m_scraps: project overview

Raymundo Arroyave

2026-10-19

Contents

m_scraps: project overview **1**
1. What SCRAPS is . . . 1
2. What m_scraps adds and fixes . . . 3
3. "Is SCRAPS lucky, or is it actually optimizing?" . . . 9
4. The Python tool suite (`tools/`) . . . 9
5. Test suite . . . 11
6. Independent validation and matched benchmarking . . . 12
7. Release history . . . 12
8. Release scope and remaining scientific limitations . . . 13
Submission benchmark . . . 14
Evaluation of supplied structures . . . 14
Completed submission benchmark . . . 14

m_scraps: project overview

This document describes (a) what SCRAPS is, (b) what was extended in this fork, and (c) how to use the resulting tool. It is meant as a technical map of the codebase for a new collaborator or for our future selves. For installation and quick-start usage see the top-level `README.md`; for benchmarks against ATAT mcsqs see `docs/mcsqs_comparison.md`.

---

1. What SCRAPS is

SCRAPS — **S**uper**C**ell **R**andom **AP**proximate**s** — is a combinatorial-optimization code for generating special quasi-random structures (SQS) and short-range-ordered (SRO) supercells of multicomponent alloys. The upstream version was developed at Iowa State by Rahul Singh, Prashant Singh, Aayush Sharma, Ganesh Balasubramanian, and Duane D. Johnson, and is published in:

R. Singh et al., *Nature Computational Science* **1**, 54 (2021). DOI: 10.1038/s43588-020-00006-7

The upstream repository is `DuaneDJohnson/Hybrid-Cuckoo-Search`. The MIT license has been preserved with dual copyright (Iowa State + Arroyave fork).

1.1 The Warren–Cowley formulation

For a multicomponent alloy on a single sublattice, the Warren–Cowley short-range-order parameter is

$$\alpha_{ij}^{(k)} = 1 - \frac{P(j \mid i,k)}{c_j}$$

where $P(j \mid i,k)$ is the conditional probability that the neighbor of an atom of species i at the k-th coordination shell is of species j, and c_j is the bulk concentration of species j. The "random" limit (an exact SQS) has $\alpha_{ij}^{(k)} = 0$ for all (i,j,k). Ordered structures (B2, $L1_2$, etc.) have characteristic non-zero α signatures at specific shells.

SCRAPS' loss function on a candidate supercell is

$$E = \sum_{k=1}^{K_{\max}} w_k \sum_{i \neq j} \left(N_{ij}^{(k)} - T_{ij}^{(k)}\right)^2$$

where $N_{ij}^{(k)}$ is the actual count of (i,j) pairs at shell k in the candidate and

$$T_{ij}^{(k)} = N_i \cdot (1 - \alpha_{ij}^{(k),\,\text{target}}) \cdot z_k \cdot N_j/N$$

is the target. The fitness reported in the log is $\sqrt{E}$.

1.2 The Hybrid Cuckoo Search

SCRAPS minimizes E over the discrete space of atomic arrangements using a metaheuristic combining:

- **A population of "nests"** (candidate supercells), each independently evolving through MC swap moves (`empty_nests`).
- **A "cuckoo" step** that probabilistically replaces low-fitness nests with new randomized ones (`get_cuckoos`), giving the search a way to escape local minima.
- **Global ranking** across MPI ranks at the end (`get_best_nest` plus the merge step in `main`).

Per-rank MC swap moves are: pick two atom positions, swap their species, evaluate fitness, accept if it improved (with a small ε-band stochastic acceptance to allow neutral moves).

1.3 Original assumptions and limitations

- Single sublattice with uniform per-shell coordination.
- A **single scalar α target** applied to every species pair at every shell (no per-pair targeting).
- POSCAR + INPUT file format (a SCRAPS-specific dialect of VASP POSCAR with no species labels, plus a custom INPUT keyword block).
- MPI-parallel over nest population; no shared memory inside a rank.

---

2. What m_scraps adds and fixes

Six bug fixes from the upstream code, a feature extension (per-pair SRO targets), spectator-sublattice support for HEB-class structures, user-facing heuristics, an algorithmic upgrade (delta-fitness MC), a reproducibility layer, and a Python-side tool suite.

2.1 Bug fixes shipped from upstream

#	Location	Description
1	`readPOSCAR`	Uninitialized `totalatoms` (upstream issue #3).
2	`fitnessvalue`	The pair-gating test `nest[center] != nest[surr]` compared species indices stored at *atom* indices, dropping unrelated entries. Replaced with `center != surr`.
3	`printtoscreen`	Wrong Warren–Cowley formula (off by ~N_i, asymmetric in $i \leftrightarrow j$); the printed α didn't match the residual the optimizer was minimizing.
4	`main`	`WEIGHTS` size mismatch claimed to STOP but read past the array end. Now `MPI_Abort` with a clear message.
5	`randomizeLattice`	Infinite loop when `sum(ELEMENT)` ≠ `Cell_DIM` × `atoms_per_basis`. Now guarded with a diagnostic and abort.
6	`Lattice::neighbourMatrix`	Per-atom shell numbering used a `removeduplicates` call inside the outer loop, so floating-point jitter near a shell boundary could assign the same physical neighbor to different shell indices for different atoms. Rewritten with **global shell numbering** at a 2% relative-gap tolerance.

Item #2 (the species-index gating bug) was the silent showstopper — it caused the upstream fitness function to misclassify ~40 % of pairs depending on memory layout, while still appearing to "work" because the resulting structures looked nominally random.

2.2 The headline feature: per-pair SRO targets

New INPUT keyword:

```
SRO  <species_i>  <species_j>  <shell_index>  <alpha_target>
```

Sets $\alpha_{ij}^{(k)\,\mathrm{target}}$ for one (i, j, k) triple. Lines may repeat; entries are automatically symmetrized $\alpha_{ji} = \alpha_{ij}$. Pairs not mentioned default to $\alpha = 0$ (the SQS target).

The targetSROpair tensor is threaded through `fitnessvalue`, `fitnessofnests`, `get_best_nest`, `empty_nests`, `empty_nests_fast`, `get_cuckoos`, and `main`. Upstream only supported a single scalar α applied uniformly; this fork lets each species pair at each shell have its own target.

This is what makes targeting **classical ordered structures** in a single SCRAPS run possible. See `tests/{b2_bcc_binary,l12_fcc_a3b, l10_fcc_ab,d03_bcc_a3b,l11_fcc_ab,l21_bcc_x2yz}/` for the six worked cases (B2, $L1_2$, $L1_0$, $D0_3$, $L1_1$, $L2_1$/Heusler).

2.3 Spectator sublattice (HEB-class structures)

For structures like AlB_2-type (Hf,Zr,Ti,Ta,Nb)B_2 high-entropy diborides, one sublattice is compositionally fixed (boron) and the other carries the chemical disorder. New INPUT keyword:

```
SPECTATOR_BASIS_SPECIES  <basis_index>  <species_index>
```

declares that all atoms at the given POSCAR basis position are fixed at the given species and not touched by MC. The fitness function becomes "M-sublattice only" — pairs involving a spectator atom are excluded, the reference concentration becomes N_j^M/N^M, and the effective coordination is the count of non-spectator neighbors per non-spectator atom.

A coordination check (`Lattice::coordination_per_shell`) is run at startup on the non-spectator subset; if it's non-uniform, SCRAPS aborts with a diagnostic pointing to ATAT mcsqs for the truly multi-sublattice cases SCRAPS can't handle.

See `tests/heb_5metal_random/` and `tests/heb_5metal_csro/` for worked HEB cases.

2.4 User-facing heuristics

Keyword	Purpose
`AUTO_MCSTEP <k>`	Override `MCSTEP` with `max(50, k · N)` once supercell size is known, removing the manual tuning. Default factor 7.
`EARLY_STOP_PATIENCE <n>`	Stop the outer optimization loop once the per-rank best fitness has not improved by more than `EARLY_STOP_TOL` for `n` consecutive iterations. Default 0 = disabled.
`EARLY_STOP_TOL <tol>`	Improvement threshold for the patience counter. Default 1e-6.
`FAST_SWAPS <0|1>`	Delta-fitness MC path (see 2.5). Default 1.
`TARGETED_SWAPS <0|1>`	Residual-biased swap proposals (see 2.5). Default 0 (opt-in).
`TARGETED_SWAP_PROB <p>`	Bias level for targeted swaps. Default 0.7.

Keyword	Purpose
`RNG_SEED <n>`	Reproducible per-rank seeding via $n \oplus (\text{rank} \cdot \phi_{64})$. Default 0 = use `std::random_device`.
`RANDOM_ONLY <0|1>`	Skip the MC loop and return the best of the initial random nest pool. Default 0.
`CONVERGENCE_LOG <path>`	Rank-0 per-iteration CSV log: `iteration, best_fitness, wall_seconds`. Iteration −1 is the pre-MC baseline.
`INIT_NEST <path>`	Warm-start: read a plain-text file of species indices (one per line, in basis-OUTERMOST atom order) and seed `nests[0]` on every rank with that nest. The MC then explores around the seed instead of from a uniform random init. Useful for steep multi-pair targets ($L2_1$ etc.) and iterative-refinement workflows. Default empty = pure random init.

The first three let users get to a sensible result without per-cell tuning. The reproducibility / ablation knobs (RNG_SEED, RANDOM_ONLY, CONVERGENCE_LOG) exist primarily to make the optimizer *auditable* — see §3. `INIT_NEST` lets users explicitly hand the optimizer a candidate structure as a starting point.

2.5 Algorithmic upgrades

Delta-fitness MC (`empty_nests_fast`, default on). The legacy `empty_nests` recomputed the full $O(N \cdot z)$ fitness on every swap proposal. The fast path precomputes the constant target matrix $T_{ij}^{(k)}$ once per nest, maintains the pair-count tensor $N_{ij}^{(k)}$ as state, and applies an $O(z)$ delta per swap (touching only the neighbors of the two swapped atoms; the swapped pair itself contributes zero net delta by symmetry). On RNG-pinned A/B tests against the legacy path, this yields **2–18× speedup on loop-bound cases** with identical final fitness:

Case	Legacy wall	Fast wall	Speedup
25:50:25 n=108 (plateau)	15.2 s	0.8 s	18.6×
25:50:25 n=256	3.1 s	0.4 s	8.2×
BCC 50:50 n=54	4.8 s	0.6 s	7.4×

Targeted swap proposals (`empty_nests_fast`, off by default). With probability `TARGETED_SWAP_PROB`, propose swaps biased toward the species pair (i, j, k) with the largest weighted residual[2]. The remaining fraction stays uniform-random for exploration. **Honest finding from our benchmarks: on the test cases we measured (cell sizes up to n=500, B2 / $L1_2$ / $L1_0$ / $D0_3$ / $L1_1$ ordered targets, infeasibility plateaus), uniform-random already converges fast enough that the bias adds overhead without speedup.** Kept as an opt-in for workloads where uniform random would genuinely struggle (very large supercells, complex multi-pair SRO, highly non-convex landscapes) and as a starting point for further algorithmic experimentation.

2.6 Feasibility check (the "is fitness=0 even achievable?" math)

For an SQS target ($\alpha = 0$), the expected pair count $T_{ij}^{(k)} = N_i \cdot z_k \cdot N_j/N$ must be **integer** for every weighted shell-pair. If it isn't, the fitness has a strictly positive lower bound — the optimizer plateaus at a non-zero value no matter how long it runs.

This caught us by surprise during the cell-size benchmark: 25:50:25 on FCC k×k×k is feasible for k even and infeasible for k odd (at shell 1, $T_{AC} = 3k^2/2$), explaining the deterministic `fitness = √2.5 approximately 1.58` plateau at n=108 and n=500.

`tools/suggest_cell.py` walks candidate cell sizes (including non-cubic supercells) and returns the closest feasible `Cell_DIM`, plus the fitness floor for infeasible candidates. It handles HEB-style spectator sublattices via the same M-only math SCRAPS uses internally. The C++ runtime also runs the feasibility check at startup and prints a warning naming the offending pair(s) when the configured `(ELEMENT, Cell_DIM, MAXSHELLNUM)` is infeasible.

2.7 Multi-sublattice SQS (the v0.4.0 headline)

For structures with two or more chemically-disordered sublattices (rocksalt HEA-carbides, double perovskites, full Heusler with independent X/Y/Z mixing, spinels), upstream SCRAPS had no API for designing per-sublattice compositions — `ELEMENT` is a single flat species count over all positions and `randomizeLattice` did a flat shuffle. The fork adds a `SUBLATTICE` INPUT keyword that partitions basis positions and species into disjoint sublattices, with every swap-proposal site gated so atoms only swap within a sublattice.

```
SUBLATTICE <id> BASIS <b1> <b2> ... SPECIES <s1> <s2> ...
```

Disjoint species per sublattice means `pairGr[i][j][shell]` and `targetSROpair[i][j][shell]` need no new dimension — cross-sublattice pair counts (e.g. metal–carbon nearest-neighbor in HEA carbides) naturally fall in the right bins because species `i` and `j` only live on one sublattice each.

INPUT-level validation in the parser (disjoint species, disjoint basis, in-range, non-empty blocks); POSCAR-level validation in `main.cpp` (BASIS bound, spectator-vs-sublattice conflict, ELEMENT sum matches per-sublattice capacity, full basis coverage). When no `SUBLATTICE` keyword is given, `main.cpp` synthesizes a default single-sublattice spec covering all non-spectator basis positions and species, so every downstream path (randomizeLattice, all five swap-proposal sites, the targeted-swap residual scorer) uses one uniform code path regardless of single- vs multi-sublattice mode — no `if/else` branching. The legacy `SPECTATOR_BASIS_SPECIES` for fixed-species sublattices continues to work alongside (and is, in effect, a special case of "sublattice with one allowed species").

Sublattice detection helper: `tools/sublattice_detect.py` reads a POSCAR via pymatgen, runs Wyckoff-orbit analysis (spglib under the hood), and emits draft `SUBLATTICE` blocks with `<fill in>` placeholders for SPECIES. Two basis positions in the same Wyckoff orbit have identical pair-correlation environments, so the orbit partition is the unique coarsest partition that respects the lattice symmetry — the right default for multi-sublattice SQS. Users can override the auto-detected partition by writing `SUBLATTICE` blocks by hand (e.g. to break symmetry intentionally).

Validated against rocksalt and Heusler $L2_1$ POSCARs: `sublattice_detect.py` reproduces the textbook orbit assignments (NaCl → 4a + 4b; Cu_2MnAl → 4a Mn + 4b Al + 8c Cu).

2.8 MPI singleton-mode diagnosis

The macOS-with-Homebrew-Open-MPI setup commonly fails to form a real multi-rank communicator when `mpirun` from a different MPI install (e.g. anaconda's MPICH Hydra `mpirun`) is first on `$PATH`. Each process then runs as a singleton with rank 0 and `nproc=1`, the 4 "ranks" do independent searches, and only one's `SCRAPS.vasp` survives (the last to write).

Three coordinated fixes: 1. **C++ runtime detects the condition** (`nproc==1` but `$OMPI_COMM_WORLD_SIZE > 1`) and prints a loud one-shot warning with the fix. 2. **`tools/tests` and `tools/run_sqs.py` auto-resolve** the matching `mpirun` by looking next to the `mpic++` from `build.sh` (`$MPICXX`) before falling back to PATH. 3. **README section** documenting the gotcha.

When real multi-rank can't be achieved, the conv-log path is suffixed with the OS PID so concurrent singletons don't race-and- interleave their writes (the test helper handles both modes).

2.9 Opt-in higher-order figure objective

`tools/higher_order.py` uses icet 4.x to enumerate decorated symmetry orbits, map their periodic figures to SCRAPs site order, and export point-function values in the API's species order. Supported figures have order 2–4, with one or more disordered sublattices and optional fixed spectators. Shared species are supported with explicit per-group integer counts in figure mode. Each local point function gets a global basis row, with zero entries outside its allowed species set; each term references the row appropriate to its site. Concentration moments use the site's local species fractions. Symmetry enumeration preserves declared sublattice identities, including groups with identical allowed physical species. icet removes fixed-site constant factors rather than introducing redundant chemical figures; fully fixed sites remain present in the geometry and occupation constraints. The exporter validates its explicit products against icet cluster vectors before writing. Random-alloy targets are products of concentration-weighted point-function moments; they are not assumed zero and do not correct for finite-cell sampling without replacement.

`SCRAPs/FigureObjective.hpp` reads `FIGURES_FILE` and computes orbit averages. Each term stores a coefficient and `(site, point-function)` factors; coefficients sum to one within each decorated component. Periodic images that map to the same site remain repeated factors, rather than being collapsed into a lower-order cluster. A site-to-term incidence list enables swap updates. The union of both sites' incidence lists visits shared terms once; rejected moves leave accepted state untouched. Full recomputation refreshes the incremental state every 1,024 proposals and at final scoring.

`main.cpp` routes every global fitness call and local search kernel to this backend when enabled. It retains the native cuckoo population/MPI gathering and the SA/PT acceptance and temperature machinery. Compound moves compose swap updates before a joint decision; `FAST_SWAPS 0` supplies a full-evaluation reference. `RNG_SEED` seeds a separate reproducible figure-kernel stream. A separate best-evaluated archive survives SA uphill moves and PT exchanges without changing chain state. Figure-mode cuckoo proposals accept improvements or numerical ties (1e-12), rather than applying the legacy pair-count error band to normalized correlations. The legacy pair path remains the default. Pair-specific `TARGETED_SWAPS`, and `SRO` fail clearly in figure mode. Swaps use the existing same-sublattice site pools and skip fixed spectators. Randomization preserves each local count. Legacy pair-coordination uniformity is not required by the figure objective. The existing pair neighbor graph is still built, but figure mode suppresses the legacy SRO diagnostic; use `FIGURE_CORRELATIONS.csv` for its actual targets.

The input controls are FIGURES_FILE, FIGURE_OBJECTIVE (weighted_l1 or atat), FIGURE_WD (default 0), FIGURE_WN (default 1), FIGURE_WR (default 1), and FIGURE_TOL (default 0.001). All are validated and known to the INPUT parser. In normalized weighted-L1 mode, $w_\alpha = \exp(-w_d d_\alpha/d_0) w_n^{n_\alpha - 2}$ applies to every decorated component, including pairs. One global denominator preserves relative order penalties. No extra multiplicity weighting or per-order normalization is applied. Geometric decay is a prior for interaction importance, not a property-error guarantee.

FIGURE_CHECKPOINT_SECONDS seconds enables atomic per-rank best-figure snapshots (FIGURE_CHECKPOINT.rankN.j containing occupations, correlations and objective score. Default 0 disables output; positive finite values specify the minimum interval between snapshots at proposal boundaries. Requires FIGURES_FILE. Snapshots preserve the search RNG and annealing schedule; they are assessable structures, not restart state. Benchmark readers must timestamp observation, including startup, and retain snapshots observed before their deadline.

ATAT mode reproduces the 3.52 calc_objective_func prescription: initialize one range per order to its largest included diameter plus d_0, truncate at the first mismatched diameter, cap each higher-order range by the preceding order, and normalize residuals over diameters at least the minimum range minus ATAT's zero_tolerance=0.001. Subtract $w_r \sum_{p=0}^{n_{max}-2} w_n^p R_p/d_0$ from this error. Compatibility also retains ATAT's −FLT_MAX sentinel when the unnormalized weighted residual sum is below 0.001. This can arise through very small weights despite unmatched correlations. Final reporting therefore evaluates maximum residual independently; negative/sentinel fitness never drives the old zero-error termination check. Cutoff completeness and the icet point basis must also match before comparing to an ATAT run: objective compatibility alone is not identical feature-space compatibility.

The version-1 text format remains supported for unconstrained systems and is whitespace-delimited (no comments):

```
SCRAPS_FIGURES 1
N_sites N_species N_basis_rows N_components
<9 cell values, row-vector order>
<N_sites fractional xyz positions, SCRAPs basis-major z/y/x order>
<N_basis_rows rows of N_species point-function values>
<for each component: order diameter target N_terms>
<for each term: coefficient site_0 basis_0 ... site_(order-1) basis_(order-1)>
```

Constrained exports use SCRAPS_FIGURES 2. Immediately after the basis matrix, version 2 inserts one row per site: N_allowed species_0 .... The native reader checks these allowed sets against the SUBLATTICE and SPECTATOR_BASIS_SPECIES declarations after expanding the lattice, and rejects version 1 for constrained systems. Python validates full site/species coverage, disjoint basis sets and integer counts against each group's capacity before export. The JSON additionally records basis/site allowed sets and local basis moments. Explicit local counts use format version 3. After the allowed-species rows, each site has group_index count_0 ... count_(TYP-1): active groups use INPUT declaration order and their full local budget; fixed sites use group −1 and a one-hot occupation count. The native reader checks this metadata against INPUT, rejecting stale targets after a local-composition or partition change. For overlapping species the exporter uses distinct auxiliary alphabets only during symmetry enumeration, preserving physical atomic-number ordering within each group. Exported occupations and structures use physical species. For shared species, append COUNTS n0 n1 ... to every SUBLATTICE line, aligned with its SPECIES list. These local budgets plus fixed occupations must sum exactly to ELEMENT. The Python API uses four-member tuples (id, basis_positions, species_indices, counts); CLI partition JSON follows the same layout. The native optimizer initializes and conserves each local budget. Without

explicit counts, disjoint-species behavior is retained. Legacy Warren–Cowley pair mode rejects this extension because its pair reference arrays do not resolve shared species by sublattice.

The C++ reader checks dimensions, finite values, indices, coefficient normalization, presence of pairs, and consistency of the cell/site ordering with POSCAR and `Cell_DIM`. The exporter writes a companion JSON containing provenance and feature metadata. Final component correlations and native occupations are saved separately for independent reconstruction. Performance and scientific superiority over pair-only optimization remain to be measured.

3. "Is SCRAPS lucky, or is it actually optimizing?"

The reproducibility / ablation keywords in §2.4 exist mainly to answer this question rigorously. The experiment:

Case	metric	Random-only (best of 96)	SCRAPS (full MC)	improvement
25:50:25 n=108 SQS plateau	fitness	~20 ± 3	1.58 (all 3 seeds)	**~12×**
25:50:25 n=108	max|α|	0.14–0.26	0.012	**~15×**
B2 n=128 ordered	fitness	369–403	0 (perfect B2)	∞
B2 n=128	search-space prob	$\sim 10^{-35}$ per nest	finds in ~1 s	qualitative

The B2 case is the clincher: random search has $\sim 10^{-37}$ chance per random nest of containing a B2 ordering. SCRAPS finds it deterministically. The MC search is doing real combinatorial optimization, not just shuffling.

Reproducible via `docs/baseline_vs_scraps.sh`.

---

4. The Python tool suite (`tools/`)

Script / module	Purpose
`verify_sro.py`	Independent pymatgen-based α-matrix verifier. Supports multi-shell, spectator-aware HEB mode, target-file pass/fail comparison, JSON output for CI. Used as the cross-check on every `SCRAPS.vasp` we generate.
`suggest_cell.py`	Picks the smallest feasible Cell_DIM near a user-supplied target atom count for a given composition. Supports cubic-only or non-cubic search, HEB spectator sublattices, and diagnose mode for a single `(ELEMENT, Cell_DIM)` configuration.

Script / module	Purpose
`run_sqs.py`	One-shot wrapper: composition + target atoms → suggest_cell pick → INPUT with sensible heuristic defaults → mpirun → result. Handles HEB targets via `--spectator-basis-species`.
`scraps_ase.py`	ASE interoperability: `run_sqs_from_atoms` takes an ASE `Atoms` primitive in and returns an SQS `Atoms` out; `read_scraps_output` / `write_scraps_poscar` / `make_scraps_inputs` are lower-level helpers. Handles the SCRAPS vs canonical-VASP POSCAR layout difference internally.
`sublattice_detect.py`	Wyckoff-orbit-based sublattice detection. Reads a POSCAR via pymatgen, runs `SpacegroupAnalyzer` (spglib), groups basis positions by orbit, emits draft `SUBLATTICE` blocks for INPUT. `--write` appends to an existing INPUT (with `<fill in>` SPECIES placeholders); `--json` emits machine-readable output. Defaults `symprec=1e-3`; loosen for noisy POSCARs.
`plot_convergence.py`	Plots `CONVERGENCE_LOG` fitness-vs-iteration trajectories (single or multi-run overlay).
`scraps_wizard.py`	Novice-friendly CLI orchestrating the other tools end-to-end. Three subcommands: `quick --template <name> --composition X:0.5,Y:0.5` one-shots from a built-in template (FCC/BCC/B2/rocksalt/Heusler-L2_1/perovskite-ABO_3); `from-structure FILE` reads any POSCAR/CIF, auto-detects Wyckoff orbits via `sublattice_detect.py`, prompts for per-orbit composition; `interactive` is a step-by-step REPL with prompts. All paths set `AUTO_BUDGET=2`, pick a feasible cell from `--target-atoms`, call `run_sqs_from_atoms`, and export to CIF / VASP / XYZ.

Script / module	Purpose
`scraps_htp.py`	High-throughput composition-space sweeps for HEA exploration. Three subcommands: `linear` (binary alloy along composition axis), `simplex` (N-ary on the Gibbs simplex at integer resolution k – C(k+n-1, n-1) compositions), and `list` (explicit compositions from CSV/JSON). Reuses `scraps_wizard.run_pipeline` per composition; writes one subdir per composition + a top-level `manifest.json` summary. Supports `--workers N` for parallel runs and resumes by skipping compositions whose `sqs.cif` already exists. Multi-sublattice templates accept `--fixed ORBIT_INDEX:Species:Frac` to pin non-swept orbits.

All are pymatgen-and/or-ASE-based and run completely outside the C++ code. `verify_sro.py` is in particular the canonical "did SCRAPS actually achieve what it claims?" cross-check.

---

5. Test suite

`tools/tests/`, run with `pytest`, contains native, Python and frozen-oracle regressions. Native integration tests generate their structures in scratch directories. Optional icet-export tests run in a separate CI step; frozen-oracle tests require no external engines. Search tests pin `RNG_SEED`; reproducibility also depends on build and MPI settings. The original test groups include:

- `test_verify_sro.py` (10 tests): verifier against analytically-known structures (B2 NiAl, $L1_2$ Ni_3Al, random binary, HEB random and CSRO).
- `test_suggest_cell.py` (16 tests): feasibility math across cubic and non-cubic FCC/BCC supercells, HEB spectator-aware cases.
- `test_run_sqs.py` (4 tests): end-to-end smoke + INPUT generation.
- `test_scraps_optimization.py` (10 tests): full mpirun-based optimization regressions — feasible SQS hits 0, infeasible plateau hits the predicted floor, B2 SRO target reaches max ordering, RANDOM_ONLY worse than SCRAPS, FAST_SWAPS matches legacy fitness, convergence trajectory is monotone non-increasing, spectator HEB completes correctly.
- `test_scraps_ase.py` (8 tests): ASE I/O round-tripping and the end-to-end ASE wrapper on FCC SQS and BCC B2.
- `test_input_parser.py` (13 tests): INPUT keyword parser hardening — required-keyword aborts, ELEMENT-vs-TYP cross-check, Cell_DIM validation, comment/blank-line handling, unrecognized-keyword warnings.
- `test_poscar_parser.py` (12 tests): defensive VASP POSCAR parser on every dialect (scale-as-volume, species-label line, Selective dynamics, Cartesian vs Direct, multi-species counts).
- `test_warm_start.py` (7 tests): `INIT_NEST` warm-start round-trips and preserves an already-optimal nest.
- `test_sa_kernel.py` (6 tests): `MC_KERNEL=sa` reaches zero on feasible SQS, plateau matches cuckoo on infeasible, PT exchanges fire on multi-rank, PT degrades gracefully on single-rank.

- `test_cluster_swap.py` (4 tests): `CLUSTER_SWAP_K>1` reaches zero on feasible SQS, matches K=1 plateau on infeasible (negative finding for $L2_1$), default K=1 unchanged.
- `test_plot_convergence.py` (4 tests): convergence-log plotter smoke + multi-run overlay.
- `test_sublattice_parser.py` (12 tests): `SUBLATTICE` INPUT parser — round-trip, section-order independence, disjointness errors, range errors, structural errors.
- `test_sublattice_detect.py` (8 tests): Wyckoff-orbit detection on FCC, rocksalt, Heusler $L2_1$; JSON output; `--write` to INPUT; symprec sensitivity.
- `test_multi_sublattice.py` (8 tests): end-to-end multi-sublattice optimization — banner appears, partition is preserved in output POSCAR (per-position species/sublattice consistency), BASIS bound abort, ELEMENT-vs-capacity abort, spectator-vs-sublattice conflict abort, partial-coverage abort, spectator + sublattice coexist, default-synthesis path still reaches fit=0.

End-to-end tests skip gracefully when the scraps binary or mpirun isn't available, so a fresh checkout passes `pytest` even before `./build.sh`. CI (`.github/workflows/test.yml`) installs `openmpi-dev` and `libeigen3-dev`, builds scraps, and runs the full suite on Ubuntu against Python 3.11 and 3.12 — so the regression net catches optimizer changes, not just verifier-math changes.

---

6. Independent validation and matched benchmarking

The completed submission benchmark compares native cuckoo search, native SA, ATAT 3.52 mcsqs and icet 4.0 with a matched ATAT objective on the same pair, triplet and quadruplet targets. It includes BCC and rocksalt, 128/432 sites, random and attainable nonzero targets, and 40 fixed trials with 30-second, four-core budgets. See the completed-benchmark section below and the README for the full protocol and limitations.

Frozen ATAT outputs also validate the native C++ evaluator, incremental swaps, the independent Python evaluator and the new structure-analysis workflow. The diboride, MAX-211 and Heusler cases include binary-equiatomic SRO checks obtained from external pair correlations. Users can rerun these with their own corrdump installation; ordinary tests need neither ATAT nor icet.

`docs/mcsqs_comparison.md` retains historical experiments with different objectives and settings. Their timing ratios are not evidence for a general speedup or an inherent accuracy limit of another method.

---

7. Release history

- **v0.4.2** (this commit) — Per-Wyckoff-orbit SUBLATTICE on lattices with non-uniform coordination (fluorite, sigma phase, Laves, MAX phases). Generalized `Lattice::coordination_per_shell` to check uniformity WITHIN each sublattice rather than across all non-spectator atoms — different sublattices may legitimately have different z (fluorite cation z=8 vs anion z=4). The per-sublattice fitness machinery from v0.4.1 already handled the math; this commit just lifts the safety check. Feasibility-warning math also generalized to per-sublattice. Tests: 4 new fluorite cases in `tools/tests/test_non_uniform_coordination.py` (fluorite without SUBLATTICE still aborts; with per-Wyckoff SUBLATTICE runs cleanly; SQS quality and CSRO targeting both work). 126 tests.
- **v0.4.1** — Per-sublattice c_j fitness, B2 mcsqs comparison, multi-sublattice tutorial notebook. The fitness fix swaps the global N_j/N_M reference for the per-sublattice N_j/N_τ reference

+ per-(σ,τ,k) coordination tensor; without it the multi-sublattice optimizer minimizes the wrong target and drives toward clustering rather than per-sublattice SQS. Single- sublattice and HEB-spectator cases reduce to the prior formula exactly. Validated on (Ni,Co)(Ti,Zr) B2 50/50 in `docs/compare_b2_multisub.py`: SCRAPs converges to exact α=0 on intra-sub shells in <1 s for the SQS case (mcsqs leaves max |α| approximately 0.1–0.2 in 80–130 s on the same problem); SCRAPs hits the per-pair α targets exactly for ordered (α=−1), clustered (α=+0.5), and mixed CSRO patterns at both N=4 and N=6 in under 1.3 s. Tutorial: `notebooks/04_multi_sublattice.ipynb`. 122 tests.

- **v0.4.0** — multi-sublattice SQS support (`SUBLATTICE` keyword + Wyckoff-orbit detector `tools/sublattice_detect.py`). All swap-proposal sites gated by sublattice; randomizeLattice rewritten to draw from per-sublattice budgets; POSCAR-level validation (BASIS bound, ELEMENT vs capacity, spectator-vs-sublattice conflict, full coverage). When no `SUBLATTICE` is declared, `main.cpp` synthesizes a default single-sublattice spec so legacy behavior is preserved exactly. 122 tests. **Note**: tagged only at v0.4.1 — the v0.4.0 commits are runnable but produce semantically wrong multi-sublattice output (global-c_j target drives toward clustering); the capability is correct from v0.4.1 onward.
- **v0.3.2** — K-atom cluster swap moves (`CLUSTER_SWAP_K`). Negative finding documented: K in {2, 4} also plateaus at ~148 on the $L2_1$ cold-start case (same as cuckoo, SA, PT). The barrier is move topology, not move size — `INIT_NEST` warm-start remains the recommended workaround. 94 tests.
- **v0.3.1** — Parallel-tempering MC kernel (`MC_KERNEL=pt`): per-rank log-spaced temperatures, periodic Metropolis exchanges between adjacent ranks. Plus `docs/sa_vs_cuckoo.sh` benchmark and the SA-vs-cuckoo section of `mcsqs_comparison.md`. 90 tests.
- **v0.3.0** — Warm-start API (`INIT_NEST`), simulated-annealing kernel (`MC_KERNEL=sa`), JOSS paper draft under `paper/`, delta-vs-legacy fitness disagreement fix (the $L2_1$ warm-start case where the perfect Heusler nest has zero X-Y and X-Z 2NN pairs, exposing a `countgr==0` edge case in the legacy fitness). 80 tests.
- **v0.2.2** — INPUT parser hardening: unrecognized-keyword warnings, # comment/blank-line support, required-keyword validation, ELEMENT-vs-TYP cross-check, `Cell_DIM=3` validation; dead `removedublicates` removed. 73 tests.
- **v0.2.1** — Real VASP POSCAR parser (handles full canonical VASP-5 layout, defensive abort on malformed input); ASE coordinate bugfix in `tools/scraps_ase.py`; three Jupyter tutorials (`notebooks/01..03`); `docs/OVERVIEW.{md,tex,pdf}`. 60 tests.
- **v0.2.0** — Per-pair SRO API, HEB-class spectator sublattice support, user-facing heuristics (AUTO_MCSTEP, EARLY_STOP_*, RNG_SEED, RANDOM_ONLY, CONVERGENCE_LOG, FAST_SWAPS, TARGETED_SWAPS), delta-fitness MC (2-18× speedup), feasibility check (`tools/suggest_cell.py` + C++ warning), ASE interop (`tools/scraps_ase.py`), MPI singleton-mode diagnosis and fix. 48 tests.

8. Release scope and remaining scientific limitations

The proposed release combines generation, structure analysis, worked examples, checksummed external references and a bounded benchmark. Publication of the repository, release and Zenodo archive requires the author's final approval.

The benchmark has limited repetitions and does not establish broad statistical superiority. Multi-component SRO inversion from ATAT correlations remains a future extension; current independent SRO fixtures cover binary equiatomic mixed sublattices. Structure mapping accepts small displacements

and atom reordering, with cell agreement within 0.0001 angstrom for serialization rounding; it does not fit arbitrary strain, rotations, origin shifts or vacancies. Prototype examples do not establish phase stability or property accuracy.

Submission benchmark

`tools/benchmark/submission/run.py` is the fixed four-method comparison driver. It generates eight problems (two crystal families × two sizes × two target modes), runs every method on every problem, and repeats the two smaller nonzero-target problems with a second seed: 40 trials at 30 s/four cores. `matched_mcsqs.py` provides the ATAT cluster mapping and independent audit; `icet_extension.py` retains icet 4.0's annealer with a shared ATAT score; `sro_targets.py` constructs realizable pair/triplet/quadruplet targets. The saved realization is withheld from optimizer initialization. Native checkpoint observation includes unfinished annealing schedules. The driver freezes sources and inputs, resumes only matching runs, and exports numerical regression fixtures. See README for reproduction commands and timing limits.

Evaluation of supplied structures

`tools/structure_analysis.py` implements `m_scraps analyze` and the public `analyze_structure` Python API. It maps structures to an explicit full-supercell reference, counts ASE periodic neighbors by directed sublattice pair, and normalizes Warren–Cowley SRO by the destination-sublattice concentration. Coordination is conditioned on the center species, so nonuniform coordination is reported rather than silently replaced by a global average. Optional SRO targets produce residuals and aggregate errors. Missing pairs remain undefined.

For supplied native figure files it reuses the independent Python decorated correlation evaluator and benchmark metric definitions. Species ordering, figure hash, basis, targets and weights accompany the result. Mapping requires a unique bijection within the displacement tolerance and equal cells; arbitrary strain, origin registration and vacancy fitting are outside this interface. Tests include exact B2 ordering, sublattice normalization, fixed-site violations, permuted/relaxed sites, and archived ATAT pair/triplet/quadruplet correlations.

`m_scraps compare-atat` supports offline golden-result verification and optional live `corrdump` replay. The `analysis_oracle` fixtures retain raw external stdout, ATAT inputs and checksums for three materials, with two configurations each. Live replay also accepts a new decoration of a selected reference cell. SRO checks use the documented binary-equiatomic inversion of ATAT pair correlations; they do not relabel an internal SRO calculation as an independent oracle.

Completed submission benchmark

The frozen 40-trial campaign produced 40 validated outputs, with saved ATAT oracle vectors and attainable-reference witnesses for nonzero targets. The portable corpus is `tools/fixtures/submission_benchmark`; its manifest checks all inputs and results. `m_scraps benchmark --report-only` reads it without external engines; a short native rerun uses the same figure definitions.

The CSV retains every trial, including RMS and per-order errors. The figure shows individual observations and censored threshold passages. Cuckoo reached maximum error 0.01 in 5/10 trials; each other method did so in 2/10. These are limited-seed screening observations, not a universal ranking.